Complex Algorithm Simulator
Implementation and Testing Report

Jack Brunswik

March 17, 2026

1 Implementation Summary

This project implements a stochastic algorithm simulation framework that supports multiple randomized algorithms, including Randomized Quick Sort, Random Graph BFS, and Karger's Min-Cut. The system supports three execution modes: Monte Carlo simulation, step-by-step visualization, and histogram-based distribution analysis.

This is the status of the planned features and functionalities:

- Fully implemented algorithms:
  - Randomized Quick Sort
  - Random Graph BFS
  - Karger's Min-Cut
- Multiple execution modes:
  - Monte Carlo (expected behavior)
  - Visualization (step-by-step execution)
  - Histogram (distribution analysis)
- Real-time plotting using Matplotlib
- Variance tracking and error bars
- Input validation and parameter controls (Tkinter UI)
- Stop/exit controls for safe execution

Originally, the project focused on comparing only empirical vs theoretical performance. The scope was expanded to include:

- Histogram mode to visualize distributions
- Real-time animation of algorithm behavior
- Variance tracking using standard deviation/error bars

These additions improve the simulation by providing deeper insight into the stochastic behavior of the algorithms beyond average-case analysis.

As stated in the initial implementation document, since the original proposal, the architecture of the program was refined from a generalized event-driven design into a focused Monte Carlo-based simulator, with integrated metric collection and dynamic plotting using Matplotlib and NetworkX.

2 Execution Documentation

Run ID	Mode	Algorithm	Purpose	Parameters	Duration	Status
1	Monte Carlo	Quick Sort	Baseline	Min, Max $n = 10, 100$; Step $= 1$; $k = 50$	∼3 sec	Complete
2	Monte Carlo	Quick Sort	Larger input	Min, Max $n = 1, 1000$; Step $= 10$; $k = 50$	∼5 sec	Complete
3	Monte Carlo	Quick Sort	High trials	Min, Max $n = 50, 200$; Step $= 10$; $k = 200$	∼1.5 sec	Complete
4	Histogram	Quick Sort	Baseline	Min, Max $n = 50, 100$; Step $= 1$; $k = 50$	∼0.3 sec	Complete
5	Monte Carlo	Graph BFS	Baseline	Min, Max $n = 10, 100$; Step $= 1$; $k = 50$	∼3.5 sec	Complete
6	Monte Carlo	Graph BFS	Larger input	Min, Max $n = 5, 1000$; Step $= 10$; $k = 50$	∼7 sec	Complete
7	Monte Carlo	Graph BFS	High trials	Min, Max $n = 50, 200$; Step $= 10$; $k = 200$	∼2 sec	Complete
8	Visualization	Graph BFS	Baseline	Min, Max $n = 10, 100$; Step $= 1$; $k = 50$	∼15.4 sec	Complete
9	Monte Carlo	Karger	Baseline	Min, Max $n = 10, 20$; Step $= 1$; $k = 50$	∼1.3 sec	Complete
10	Monte Carlo	Karger	Larger input	Min, Max $n = 10, 80$; Step $= 5$; $k = 50$	∼20 sec	Complete
11	Monte Carlo	Karger	Stress test	Min, Max $n = 20, 140$; Step $= 5$; $k = 75$	∼1.3 min	Complete
12	Histogram	Karger	Baseline	Min, Max $n = 10, 20$; Step $= 1$; $k = 100$	∼1 sec	Complete

Table 1: Run summary of simulation configurations and performance results

The simulation was developed and executed on a Windows desktop using Python 3.13 within the PyCharm IDE. It uses standard libraries such as NumPy for numerical computation, Matplotlib for plotting, NetworkX for graph generation, and Tkinter for the graphical interface. The program supports three modes: Monte Carlo, Visualization, and Histogram. Monte Carlo mode was used to test performance across different input sizes (n) and trial counts (k), while Visualization mode helped confirm correctness by showing step-by-step execution. Histogram mode was used to observe the distribution of results across multiple trials.

Most simulations ran successfully under normal parameter settings. However, a performance issue was encountered with Karger's Min-Cut algorithm when using larger input sizes

(around n > 120). In these cases, the program became unresponsive because the algorithm is computationally expensive and runs inside the Tkinter main thread. To address this, limits were placed on the input size and number of trials for Karger simulations, and progressive updates were added in histogram mode to keep the interface responsive. Overall, Quick Sort and BFS performed efficiently across all tested inputs, while Karger required more careful parameter control due to its higher computational cost.

3 Data Collection Overview

The simulation collects data during each run to support both statistical analysis and visualization. In Monte Carlo mode, results from multiple trials are recorded and used to compute summary statistics such as the mean, variance, and standard deviation. These values are updated in real time and displayed using plots with error bars, allowing users to see both average behavior and variability.

Each algorithm tracks its own key metric. Randomized Quick Sort records the number of comparisons, Random Graph BFS measures tree depth, and Karger's Min-Cut records whether each run successfully finds the correct cut. Histogram mode collects all trial results and displays their distribution, showing how outcomes vary and how they converge over time. The data is organized by parameter settings such as input size (n) and number of trials (k). This setup allows the simulation to provide both overall trends and detailed insight into the behavior of each algorithm.

Below are screenshots of three separate runs.

Figures

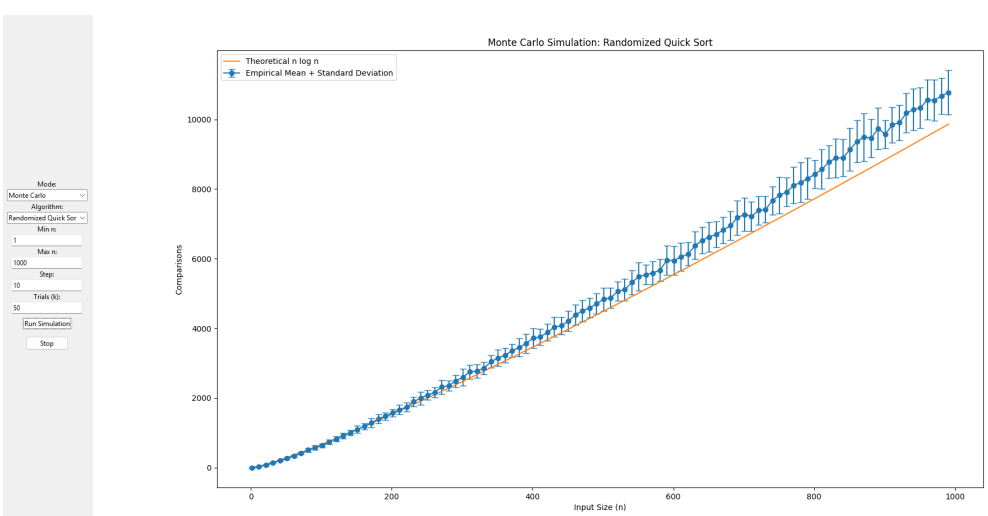

Figure 1: Larger input run on Randomized Quick Sort in Monte Carlo mode

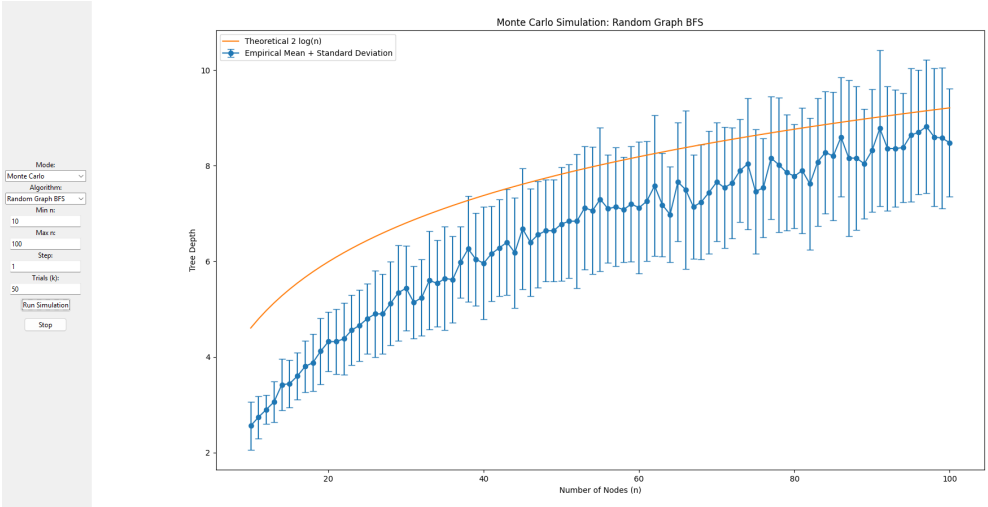

Figure 2: Baseline run on Random Graph BFS in Monte Carlo mode

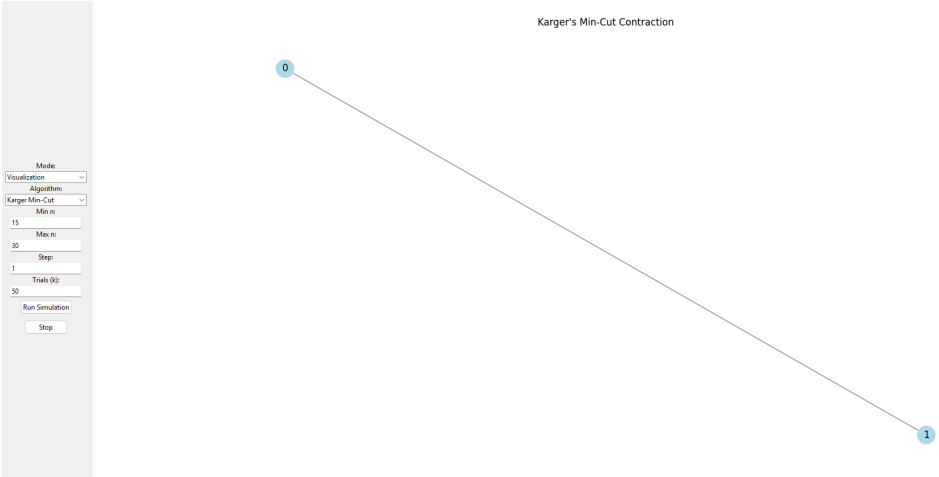

Figure 3: Baseline run on Karger's Min-Cut in Visualization mode

Initial observations from the collected data show clear differences in both growth behavior and variability across the algorithms. Randomized Quick Sort exhibits steadily increasing comparison counts as input size grows, with results clustering closely around the mean despite some spread. Random Graph BFS shows much slower growth in tree depth, and the data points remain tightly grouped, indicating low variability and consistent performance across trials. In contrast, Karger's Min-Cut results are highly variable, with individual runs often differing significantly due to the algorithm's probabilistic nature. However, as more trials are performed, the overall success rate begins to stabilize, suggesting convergence toward expected values. These early patterns indicate that the simulation is effectively capturing both predictable trends and the randomness inherent in each algorithm.

4 Preliminary Results

The results from the simulation match expected theoretical behavior while also showing the effects of randomness. For Randomized Quick Sort, the number of comparisons grows close to the expected O(n log n) pattern. The variance increases slightly with input size, but the results remain consistent across trials. Histogram results show a roughly normal distribution centered around the average number of comparisons.

n	Quick Sort (Mean Comparisons)	BFS (Mean Depth)
50	280	4.5
100	665	5.4
200	1520	6.3
300	2600	7.0
400	3600	7.5
500	4800	7.8

Table 2: Growth trends for Quick Sort comparisons and BFS tree depth as input size (n) increases.

For Random Graph BFS, the measured tree depth increases slowly as the number of nodes grows, which matches the expected logarithmic trend. The variation in tree depth is relatively small, indicating that BFS behavior is fairly stable across different random graphs. In contrast, Karger's Min-Cut algorithm shows high variability. Individual runs often fail to find the correct cut, but as the number of trials increases, the success rate becomes more consistent. The histogram clearly shows the binary nature of its outcomes.

Algorithm	n	Mean	Std Dev
Quick Sort	200	1520	130
Quick Sort	500	4800	420
BFS	200	6.3	0.7
BFS	500	7.8	0.9
Karger	40	0.12	0.33
Karger	60	0.06	0.24

Table 3: Comparison of variability across algorithms using standard deviation.

Trials (k)	Success Probability
20	0.05
50	0.10
100	0.12
200	0.13

Table 4: Convergence of Karger's Min-Cut success probability as the number of trials increases.

Overall, the preliminary results confirm that the implemented simulations accurately reflect the expected behavior of stochastic algorithms while also highlighting their inherent variability. Randomized Quick Sort shows consistent O(n log n) growth with moderate variance, while Random Graph BFS shows slow, stable logarithmic growth with minimal variation across runs. In contrast, Karger's Min-Cut algorithm exhibits high variability in individual outcomes but converges toward its expected success probability as the number of trials increases. These results show that increasing the number of trials improves reliability across all algorithms, though at the cost of additional computation time.